



中国科学院大学
University of Chinese Academy of Sciences

硕士学位论文

NP 完全性归约的人工智能辅助证明研究

作者姓名: 陶思远

指导教师: 卜东波 研究员

于春功 高级工程师

学位类别: 工学硕士

学科专业: 计算机科学与技术

培养单位: 中国科学院计算技术研究所

2026 年 6 月

Research on AI-Assisted Methods for Proving the Complexity of
Open Problems

A thesis submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Master of Engineering
in Computer Science and Technology

By
Tao Siyuan
Bu Dongbo Researcher
Yu Chungong Senior Engineer

Institute of Computing Technology, Chinese Academy of Sciences

June, 2026

摘 要

NP 完全性将大量组合问题置于统一的困难性框架中，为问题的计算难度比较和高效算法存在的可能性分析提供理论依据。证明一个问题是 NP 完全的，核心难点在于需要通过多项式时间归约证明其 NP 困难性。归约需要把任意源问题实例变换为目标问题实例，同时需要保证该映射满足多项式约束，以及源、目标实例的判定结果等价。源问题选择失当、参数变换错误等，都可能使整个证明不成立。归约的证明过程往往依赖研究者的经验和创新性构造，需要在文献检索、候选构造、实例分析和证明整理之间反复推敲。

归约证明的候选空间难以穷举，错误又常暴露在实例分析和证明展开，单纯依赖创造性和经验难以系统搜索。而人工智能擅长信息综合和搜索，适合辅助这一过程。本文将归约任务定义为程序构造问题，利用大语言模型提出候选源问题与归约映射。智能体运行与评测框架向模型提供程序执行信息、格式检查和实例判定等反馈。模型依据各阶段反馈在检索、构造、核验和证明整理之间迭代，最终得到正确的归约证明。

为考察上述方法的有效性，本文构建了一个已知归约任务的数据集。早期 132 项任务集合的实验从单次生成逐步引入基础代码智能体、初步归约评测工具和实例级判定反馈，任务通过率由 24.2% 提高到 60% 以上。这一趋势表明，面向探索过程提供相应信息和执行反馈有助于修正候选证明，为本文的方法提供了初步的有效性验证。

关键词： NP 完全性，多项式时间归约，大语言模型，代码智能体，人工智能辅助证明

Abstract

NP-completeness places a broad class of combinatorial problems within a common framework of computational hardness, providing a theoretical basis for comparing problem difficulty and examining the possibility of efficient algorithms. The central challenge in proving a problem NP-complete is often to establish its NP-hardness through a polynomial-time reduction. Such a reduction transforms every instance of a source problem into an instance of the target problem while satisfying polynomial resource bounds and preserving the YES/NO answer. An unsuitable source problem or an incorrect parameter transformation can invalidate the entire proof. Reduction proofs therefore rely heavily on a researcher’s experience and creative constructions, with repeated movement among literature retrieval, candidate construction, instance analysis, and proof development.

The space of candidate reductions is difficult to enumerate, and errors often emerge only during instance analysis or proof development. Creativity and experience alone are insufficient for a systematic search. Artificial intelligence (AI) is well suited to assist this process because of its capacity for information synthesis and search. This study formulates reduction construction as a programming task and uses a large language model (LLM) to propose candidate source problems and reduction mappings. An agent harness supplies the model with feedback from program execution, format validation, and instance decisions. Guided by feedback from different stages, the model iterates among retrieval, construction, checking, and proof development. The final output is required to constitute a correct reduction proof.

To assess the effectiveness of this approach, this study constructs a dataset of known reduction tasks. Early experiments used a set of 132 tasks and progressively introduced a base code agent, preliminary reduction evaluation tools, and instance-level oracle feedback. The task pass rate increased from 24.2% under one-shot generation to above 60%. This trend indicates that information and execution feedback aligned with the exploratory process help revise candidate proofs, providing preliminary evidence

for the effectiveness of the proposed method.

Keywords: NP-Completeness, Polynomial-Time Reduction, LLM, Code Agent, AI-Assisted Proof

目 录

第 1 章 引言	1
1.1 研究背景	1
1.2 研究意义	2
第 2 章 国内外研究现状	5
2.1 NP 困难性证明的生成与验证	5
2.2 人工智能辅助数学证明	5
2.3 NP 问题实例求解与判定反馈	6
2.4 代码智能体与运行评测框架	6
第 3 章 研究目标、内容与关键问题	9
3.1 研究目标	9
3.2 研究内容	9
3.3 总体研究路线	11
3.4 关键问题与技术难点	11
第 4 章 详细研究方案	13
4.1 NPC 归约程序构造数据集	13
4.2 基础代码智能体	15
4.3 NPC 归约评测框架	16
4.4 未知复杂度问题的 NPC 性证明尝试	17
4.5 少样本实例反馈案例	17
第 5 章 研究条件与工作计划	21
5.1 研究条件	21
5.2 研究工作计划	21
参考文献	23

术语中英对照

英文术语或专名	中文名称与本文含义
AI	Artificial Intelligence，人工智能；辅助检索、归约构造、实例分析与证明整理的方法
LLM	Large Language Model，大语言模型；根据问题定义和交互信息生成文本或程序的模型
Code Agent	代码智能体；可编辑文件、执行程序并依据结果继续交互的智能体
Agent Harness	智能体运行与评测框架；组织任务、工具、运行状态和反馈的外部环境
Oracle	实例判定器；在评价端返回问题实例 YES/NO 结果的判定程序
Skill	技能模块；向智能体提供任务知识和操作规范的说明模块
ReAct	推理与行动协同方法；使模型交替执行推理、操作和观察的方法
SWE-bench	软件工程任务基准；以真实代码库问题评价程序修复能力的基准
SWE-agent	软件工程智能体；面向代码库浏览、编辑和测试执行的智能体系统
OpenHands	软件开发智能体平台；提供终端、浏览器和受限运行环境的系统
DeepSeek	DeepSeek-V4-pro，早期消融实验使用的大语言模型

第1章 引言

1.1 研究背景

NP 完全性将大量组合问题置于多项式时间归约所定义的统一框架中，是比较问题计算难度的重要理论工具。Cook 和 Karp 建立了 NP 完全性与多项式时间归约的基本理论 (Cook, 1971; Karp, 1972)。此后，许多组合问题通过归约获得困难性结论 (Garey 等, 1979)。证明一个问题为 NP 完全问题，既要说明该问题属于 NP，也要证明其 NP 困难性。后者通常需要选择一个已知 NP 困难问题，并构造保持判定结果的实例映射。

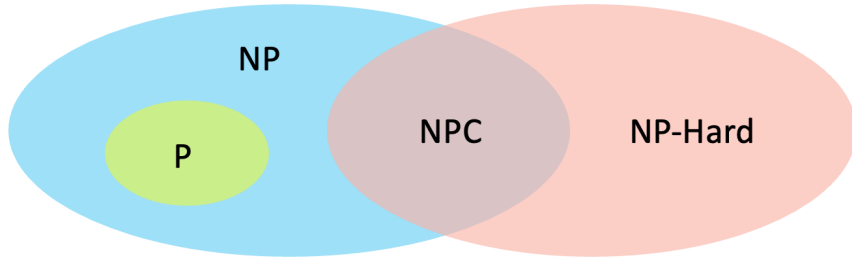


图 1.1 NP 完全性的证明条件。

Figure 1.1 Proof requirements for NP-completeness.

设 Σ 为有限字母表， Σ^* 表示由 Σ 中符号组成的所有有限字符串的集合。复杂性理论用语言 $L \subseteq \Sigma^*$ 表示判定问题。具体问题的输入对象经过编码后成为字符串，其中属于 L 的字符串称为 YES 实例，不属于 L 的字符串称为 NO 实例。将源问题和目标问题对应的语言分别记为 $L_1, L_2 \subseteq \Sigma^*$ 。若存在多项式时间可计算函数 $f : \Sigma^* \rightarrow \Sigma^*$ ，满足

$$\forall x \in \Sigma^*, \quad x \in L_1 \iff f(x) \in L_2, \quad (1.1)$$

则称 L_1 可以多项式时间多对一归约到 L_2 ，记作 $L_1 \leq_m^p L_2$ 。当 L_1 对应的问题已知为 NP 困难问题时，式 (1.1) 可用于证明 L_2 对应问题的 NP 困难性。完整证明需要给出映射 f 的构造，并分别证明 $x \in L_1 \Rightarrow f(x) \in L_2$ 和 $f(x) \in L_2 \Rightarrow x \in L_1$ 。运行时间与输出长度还须受输入长度的多项式限制。

归约构造的难点不在于复述定义，而在于寻找满足全部条件的映射。源问题

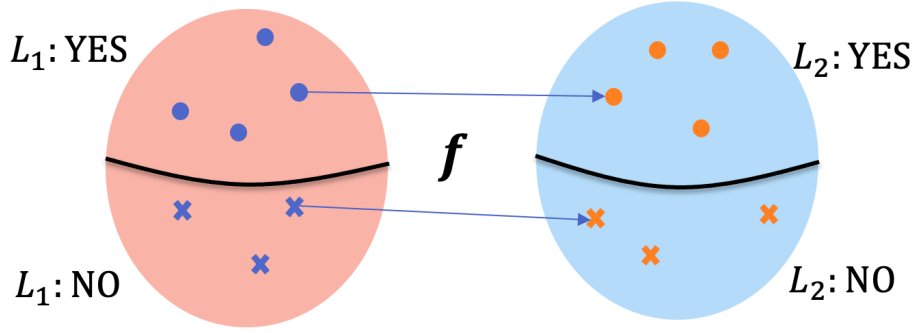


图 1.2 多项式时间归约关系。

Figure 1.2 Basic relation in polynomial-time reduction.

选择失当参数变换错误，都可能使证明失败。候选空间通常难以穷举，错误又常在边界实例分析或证明展开时才会暴露。研究者需要反复检索已有结论、比较源问题、实现候选映射、分析实例并补全证明，人工经验难以系统覆盖这一搜索过程。

人工智能在数学推理中的应用正由答案生成转向可核验的证明构造，并已用于形式化证明、欧氏几何证明和数学构造问题 (Zheng 等, 2022; Yang 等, 2023; Trinh 等, 2024; Romera-Paredes 等, 2024; Hubert 等, 2026)。这些工作表明，人工智能可以生成数学对象并依据外部反馈加以检验。但归约发现不仅需要证明给定命题，还要选择源问题并设计新的实例映射。

大语言模型可以依据问题定义和已有知识生成候选映射，为归约搜索提供起点。由于单次生成难以完成检索、构造、检验和证明修正的全过程，本课题同时结合大语言模型和智能体环境，使其依据各阶段反馈持续修正归约证明，最终以期得到正确的归约。

1.2 研究意义

本课题的首要意义在于为复杂度尚未确定的 NP 判定问题寻找新的复杂性证明。若能构造满足全部条件的多项式时间归约，便可证明目标问题的 NP 困难性；结合该问题属于 NP 的证明，还可得到新的 NP 完全性结论。这类结果能够扩充已知 NP 完全问题及其归约关系，并揭示目标问题与经典困难问题之间的结构联系。无法得到完整归约时，受限问题的困难性结论、已经证明的局部构造或能够排除某类候选映射的反例，也可以缩小后续研究的范围。

本课题也探索人工智能辅助理论证明的研究方法。现有人工智能定理证明工作多在给定命题后搜索证明 (Lample 等, 2022; Jiang 等, 2023; Xin 等, 2024), 复杂性归约还需要选择源问题并设计新的实例映射。本课题以智能体组织大语言模型, 使其依据反馈迭代构造候选, 并通过数学证明核验结果, 为人工智能参与开放式理论探索提供参考。

已知归约任务还为考察人工智能的推理能力提供了可核对的实验对象。通过改变问题结构、组合归约长度和反馈方式, 可以分析模型能否理解问题定义、建立结构对应、利用执行与证明反馈并完成多步映射。相关结果用于刻画给定模型、运行环境和计算资源条件下的能力边界, 区分可由外部反馈改善的推理环节与仍需依赖新结构发现的环节。

第 2 章 国内外研究现状

人工智能辅助多项式时间归约构造涉及归约的表达与验证、数学证明生成、NP 问题实例求解和智能体交互。现有研究已经分别处理了这些环节，但各自采用的研究对象与正确性标准并不相同。

2.1 NP 困难性证明的生成与验证

Creus 等使用 SAT 方法评价候选归约，实验中发现了纸笔构造的具体错误 (Creus 等, 2014)。这类评价依赖可编码的候选和实例，所得反例不能证明归约对所有输入成立。Zhang 等提出领域专用语言 Karp，用统一语义描述问题、实例映射和证书映射 (Zhang 等, 2022)。该语言通过代价语义约束运行时间，并用性质测试检查了 25 个教材归约；受限语法排除了部分错误，也限制了可表达的归约类型。

Grange 等研究候选归约的自动验证问题，证明部分受限表示下的验证仍不可判定 (Grange 等, 2024)。他们提出图形化的 `cookbook reduction`，并给出若干可判定子类。可判定性依赖表示形式的限制，不能直接推广到通用程序。

Karp 数据集收集了 90 个自然语言 NP 困难性归约证明，用于训练和评价语言模型 (DiCicco 等, 2025)。该数据集覆盖教材练习和论文中的归约，使源问题选择、映射构造和双向论证可在同一组材料上考察。自然语言证明中的边界条件和量词未必全部写明，评价仍较多依赖理论计算机科学专家。

Pan 等利用问题约束、知识库和多层检查构建可执行归约库，并通过归约关系图连接问题与已有求解器 (Pan 等, 2026)。结果表明代码智能体适于实现和维护成规模的已知归约，但其研究对象不包括复杂度未知问题中的源问题选择和新映射发现。

2.2 人工智能辅助数学证明

miniF2F 收集了 488 道形式化数学题 (Zheng 等, 2022)。数据覆盖 Lean、Meta-math、Isabelle 和 HOL Light 等系统。它为跨系统评价提供了统一材料，但形式化陈述已经给定。ProofNet 将本科数学教材中的自然语言命题、证明和 Lean 陈

述配对，用于研究自动形式化与证明生成 (Azerbaiyev 等, 2023)。其评价对象仍是预先选定的定理，不涉及待证结论的发现。

LeanDojo 从 Lean 项目中提取证明步骤与环境信息，并据此构建检索增强的定理证明器 (Yang 等, 2023)。其结果说明证明库检索有助于前提选择，但检索范围受已有形式化库限制。Jiang 等将非形式证明转化为形式草图，再由自动证明器补全子目标 (Jiang 等, 2023)。该方法证明非形式推理可引导形式证明搜索，但仍以给定定理和可调用的自动证明器为前提。

AlphaGeometry 用语言模型引导符号演绎程序，并通过合成数据学习欧氏几何证明 (Trinh 等, 2024)。其几何证明由符号程序检查，适用范围受几何语言和推理规则限制。AlphaProof 在 Lean 环境中通过强化学习搜索形式证明，完成了部分国际数学奥林匹克竞赛题 (Hubert 等, 2026)。逻辑内核保证了证明的正确性，但题目定义和待证结论已经固定。

2.3 NP 问题实例求解与判定反馈

NPHardEval 按复杂度类别组织 900 个算法问题，并通过动态更新题目减少测试数据泄漏 (Fan 等, 2024)。实验刻画了语言模型性能随问题难度和实例规模的变化，但评价单位仍是单个输入实例。

NPPC 提供 25 类 NP 完全问题的实例生成、求解和分析接口 (Yang 等, 2025)。持续生成的实例与答案检查程序支持规模可控的自动评价，但不产生适用于全部实例的映射函数。NP-Engine 将实例生成器、答案检查程序和启发式求解器结合，用于评价语言模型求解 NP 困难优化问题的表现 (Li 等, 2025)。合成实例的输出可自动检查，其结论仍限于给定实例上的可行解和目标函数值。

2.4 代码智能体与运行评测框架

ReAct 使语言模型交替生成文本推理和外部操作，并根据工具结果更新后续步骤 (Yao 等, 2023)。该方法表明语言模型可利用多轮环境反馈，但反馈本身不保证推理结论正确。

SWE-bench 以真实代码库中的软件问题评价模型生成补丁的能力 (Jimenez 等, 2024)。测试结果为补丁提供自动反馈，但任务通常已有测试或历史修复作为参照。SWE-agent 支持智能体浏览代码库、编辑文件和运行测试 (Yang 等, 2024)。

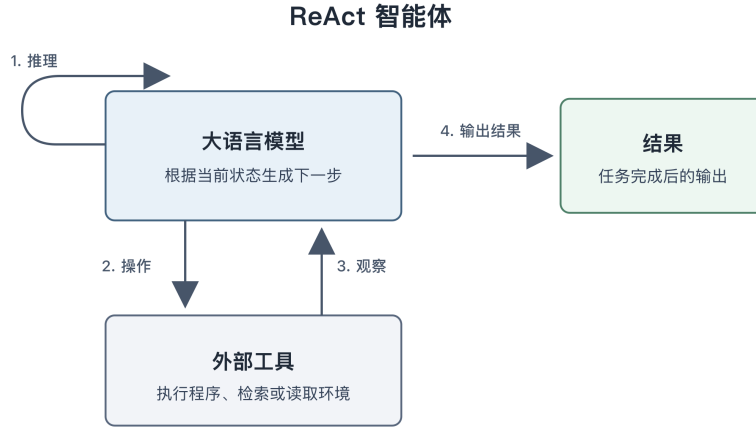


图 2.1 ReAct 智能体的基本交互结构。

Figure 2.1 Basic interaction structure of a ReAct agent.

OpenHands 提供终端执行、网页浏览和受限运行环境 (Wang 等, 2024)。这些系统使智能体得以保留中间文件并根据执行结果多次修改程序，但测试通过不能替代程序对全部输入成立的证明。

智能体运行与评测框架把复杂任务所需的工具、状态和反馈组织为连续交互。证明探索中的反馈包括候选程序的运行信息、输入格式检查、实例判定结果和证明条件的核验状态。该框架通过提供这些信息，复现研究者在源问题选择、映射构造和证明整理之间往返的过程，使人工智能辅助发现新的 NP 完全问题成为可能。

第3章 研究目标、内容与关键问题

3.1 研究目标

本课题面向复杂度尚未确定的 **NP** 判定问题，研究如何引入人工智能辅助构造多项式时间归约，并在选定问题上尝试证明其 **NP** 完全性（以下简称 **NPC**）。人工智能在该过程中不替代数学证明，而用于组织源问题检索、候选映射生成、实例反例搜索和候选修正。最终结论仍需写成可核验的复杂性证明。

设源问题和目标问题对应的语言分别为

$$L_1, L_2 \subseteq \Sigma^*, \quad (3.1)$$

其中 L_1 是已知 **NP** 困难问题， L_2 是待研究的问题。候选归约由函数 $f : \Sigma^* \rightarrow \Sigma^*$ 给出。若要证明 L_2 的 **NP** 困难性，需要证明

$$\forall x \in \Sigma^*, \quad x \in L_1 \iff f(x) \in L_2, \quad (3.2)$$

并给出 f 的多项式运行时间和输出长度上界。若还能证明 $L_2 \in \mathbf{NP}$ ，则可得到 L_2 的 **NP** 完全性。实例检验只用于发现不满足式 (3.2) 的输入，不能代替对所有输入的证明。

3.2 研究内容

现有研究尚未形成从可执行构造到未知问题复杂性证明的完整路径。自然语言归约数据集保存了证明文本，但评价仍依赖专家判断。领域专用语言和自动验证方法支持候选归约的自动检查，适用范围则受表示形式限制。**NP** 问题求解基准只检查单个实例的答案，通用代码智能体的测试结果也不直接对应归约正确性。复杂度未知的问题没有参考映射，还需处理源问题选择和新映射构造。

未知问题不存在可供对照的标准归约，若直接在该问题上开发方法，构造失败难以归因。错误既可能来自问题定义和判定程序，也可能来自映射构造或智能体的工具使用。为获得可控的研究对象，本课题先把已知 **Karp** 归约改写为程序构造任务。归约程序接收源问题实例并输出目标问题实例，判定程序在隐藏实例

上检查两者的 YES/NO 等价性。可信参考归约用于确认任务存在正确实现，对抗性实例用于暴露阈值、索引和边界条件错误。该转换过程如图 3.1 所示。数据集为归约构造提供可重复的实例级评价，也为后续分析失败原因保留统一材料。

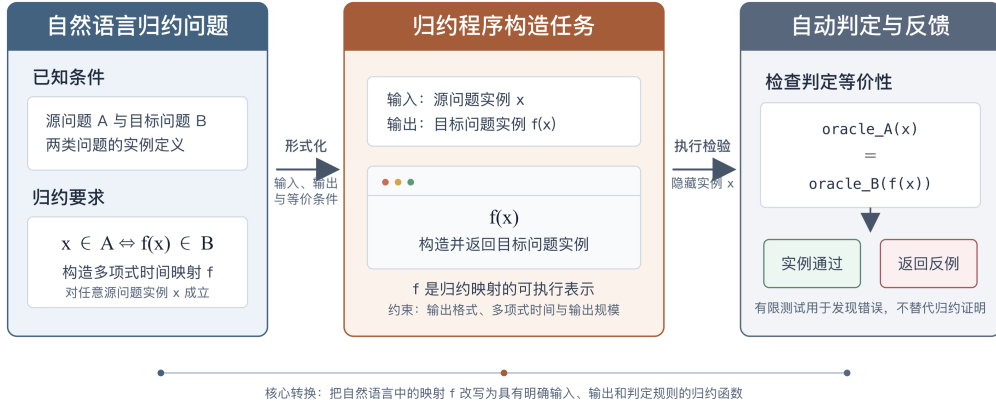


图 3.1 NPC 归约问题向程序构造任务的转换。

Figure 3.1 Conversion of an NPC reduction problem into a coding task.

归约映射的程序化表达适合由代码智能体反复编辑和执行。本课题将基础代码智能体作为阶段性任务，开发任务工作空间、文件编辑、终端执行和多轮交互功能，并完成归约任务接入、运行控制和提交接口。基础代码智能体为数据集任务和后续专用工具提供统一运行环境，但通用程序反馈无法判断目标实例格式及两类问题的判定等价性。

本课题在基础代码智能体上开发 NPC 归约评测框架。该框架通过专用工具和技能模块提供问题定义、格式校验、判定反馈和反例信息。智能体根据这些信息修改归约映射，逐步排除输出格式错误、判定不一致和规模膨胀。有限实例检验只用于发现错误，不能代替归约证明。

已知归约任务用于检验候选生成与反例修正过程，复杂度未知的问题用于考察这套方法能否支持新的理论分析。课题将选择具有明确输入编码和 NP 成员性论证的目标问题，再从已知 NP 完全问题中寻找结构相近的源问题。候选程序与失败实例将用于提取映射规则和局部性质。最终结果需要给出双向蕴含及多项式资源界的数学证明。若完整证明尚未形成，则保留适用范围明确的困难性结论、局部引理或排除某类构造的反例。

3.3 总体研究路线

总体路线如图 3.2所示。已知归约数据集提供问题定义、程序入口和实例级评价。基础代码智能体负责归约任务接入、运行控制和归约程序执行。**NPC** 归约评测框架通过专用工具和技能模块补充格式校验、判定反馈与反例信息。未知复杂度问题案例用于检验这套方法能否辅助构造新的多项式时间归约。



图 3.2 总体研究路线。

Figure 3.2 Overall research route.

数据集提供已知归约任务，基础代码智能体负责运行归约构造过程，评测框架提供归约相关反馈。归约程序通过有限实例检验后，还要改写为数学映射，并证明两个蕴含方向和多项式资源界。未知问题阶段不再以任务通过率作为最终评价，而以完整 **NPC** 证明或适用范围明确的部分理论结论为准。

3.4 关键问题与技术难点

源问题需要与目标问题具有可利用的结构对应关系。候选映射必须保持任意输入上的 YES/NO 答案，阈值、索引和边界情形都可能使归约失效。

判定程序与测试实例决定反馈是否可信。源问题和目标问题的判定结果必须准确，测试集需要覆盖 YES、NO 和典型边界实例，避免错误候选因测试不足而通过。

候选程序需要对应真正的多项式时间映射。有限测试无法排除指数搜索或先求解源实例再输出固定目标实例的做法，最终仍需证明双向蕴含、运行时间界和输出长度界。

第4章 详细研究方案

智能体辅助归约构造是本课题详细研究方案的主线，其整体过程如图 4.1 所示。智能体生成归约映射程序，执行环境运行该程序，NPC 归约评测框架检查输出格式并比较源、目标实例的判定结果，再把反例和运行错误反馈给智能体。归约程序通过实例检验后，其构造方法被整理为包含等价性与多项式界的数学证明。

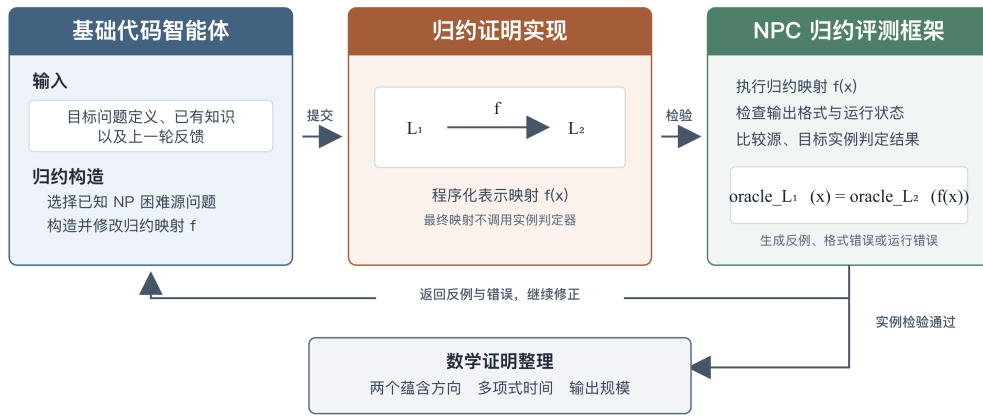


图 4.1 智能体辅助归约构造过程。

Figure 4.1 Agentic reduction construction process.

这一过程以 NPC 归约程序构造数据集提供问题定义和检验实例，以基础代码智能体提供通用执行能力，以 NPC 归约评测框架提供格式校验、实例判定和反例反馈。前三项工作共同支持复杂度未知问题的 NPC 性证明尝试。程序检验用于发现构造错误，归约正确性仍需通过数学证明建立。

4.1 NPC 归约程序构造数据集

数据集中的每个任务由源问题、目标问题、输入编码和归约程序入口组成。源问题和目标问题对应语言记为 $L_1, L_2 \subseteq \Sigma^*$ 。归约程序 f 接收源实例编码 x ，输出目标实例编码 $f(x)$ 。程序测试还要检查输出是否符合目标问题的输入格式。记 $\text{Format}_i(z) = 1$ 表示字符串 z 符合第 i 个问题的输入格式。评价端为每个问题配置实例判定程序，并记为

$$\text{oracle}_{L_i}(z) = \begin{cases} 1, & z \in L_i, \\ 0, & z \notin L_i. \end{cases} \quad (4.1)$$

oracle_{L_i} 接收问题实例的编码，返回该实例对应的 YES 或 NO 判定。源问题实例判定器用于判断 x ，目标问题实例判定器用于判断 $f(x)$ ，二者结果相同表示归约映射在该实例上保持了判定结果。实例判定器属于评价端，由评测框架在受控条件下调用，不作为映射 f 的可调用接口。即使实验设置允许智能体查询部分实例的判定结果，这些查询也只用于构造和调试，不进入最终映射。此处的实例判定器不改变多项式时间多对一归约的形式。实例级检验结果定义为

$$V_{L_1, L_2}(f, x) = \mathbf{1} \left[\text{Format}_2(f(x)) = 1 \wedge \text{oracle}_{L_1}(x) = \text{oracle}_{L_2}(f(x)) \right]. \quad (4.2)$$

归约程序超时、异常退出或输出格式不合法时，式 (4.2) 取 0。设 $H \subseteq \Sigma^*$ 为有限测试集，归约程序在 H 上的通过率为

$$\text{Score}(f; H) = \frac{1}{|H|} \sum_{x \in H} V_{L_1, L_2}(f, x). \quad (4.3)$$

该分数用于比较归约程序和反馈条件，只表示当前测试集没有发现反例。它不涉及所有输入，也不证明归约程序具有多项式时间界。数据集的定位是提供可复现的构造任务、反例和错误类型，而不是建立通用归约验证器。

当前任务分为单步归约任务 T1 和组合归约任务 T4。T1 的源问题与目标问题之间存在已知直接归约，模型需要重新构造实例映射。T4 来自多步归约路径，模型需要直接生成复合映射，用于检查中间表示、参数传递和输出规模控制。数据集规模见表 4.1。

表 4.1 NPC 归约程序构造数据集规模。

Table 4.1 Scale of the NPC reduction coding dataset.

指标	数量
归约任务总数	162
T1 单步归约任务	79
T4 组合归约任务	83

每个问题需要给出输入编码、格式检查、判定程序。用于验证归约正确性的隐藏实例集合，由固定实例（人工维护）、针对历史错误构造的实例和随机实例组成，重点覆盖边界规模、特殊结构和参数阈值。

早期 132 项任务集合已经完成不同实验设置下的消融实验。实验模型为 DeepSeek-V4-pro，每个任务设置 64 个随机测试实例。当前 162 项任务集合只作为数据集规模记录，不与早期实验结果合并统计。实验结果见表 4.2。

表 4.2 132 项任务集合上的消融实验结果。

Table 4.2 Ablation results on the 132-task set.

实验设置	全部任务	T1	T4
单次生成	24.2%	28.6%	21.7%
仅提供任务定义的代码智能体	31.8%	42.9%	25.3%
引入初步归约评测工具	36.4%	46.9%	30.1%
少样本实例反馈	60.6%	73.5%	53.0%
自由实例反馈	64.4%	79.6%	55.4%

初步归约评测工具提供归约程序执行、目标格式检查和基础自检，但不返回实例级判定结果。少样本实例反馈由评测框架返回少量固定实例上的源、目标判定结果及不一致位置，智能体依据这些实例定位归约映射中的错误。自由实例反馈允许智能体自行构造合法实例并查询相应判定结果，用于主动检查边界条件和局部构件。两种实例反馈均只用于归约构造，不把实例判定器调用写入最终映射。

结果表明，代码智能体在仅有任务定义时相对单次生成有所提升，引入初步归约评测工具后通过率继续上升，但两次增幅均较有限。少样本实例反馈带来的提升最明显，自由实例反馈在此基础上继续提升。T4 在各项设置下均低于 T1，说明组合归约中的中间编码和参数传递是主要错误来源。

4.2 基础代码智能体

基础代码智能体是归约实验的通用执行环境。该阶段开发任务工作空间、文件编辑、终端执行和多轮交互功能，并接入归约任务定义、模型配置、运行参数和提交接口。归约判定由后续评测框架负责，基础代码智能体主要保证数据集任务可以按统一方式运行。

一次实验开始时，基础代码智能体根据任务定义创建独立工作空间，写入问题说明、输入输出格式和归约程序入口。运行时设置模型、工具权限、迭代预算和资源限制。智能体在工作空间内编辑并执行归约程序，结束时通过统一接口提交最终的映射函数。判定程序与隐藏实例保留在工作空间之外，避免归约程序直接读取评价信息。

4.3 NPC 归约评测框架

基础代码智能体只能获得程序运行结果，缺少归约任务所需的语义反馈。NPC 归约评测框架在其上增加专用工具和技能模块。技能模块说明多项式归约的任务形式和归约程序约束，工具用于校验目标实例格式、调用源问题与目标问题的实例判定器，并返回判定不一致的反例。反馈包括最小化后的反例、输出格式错误或运行错误，智能体据此修改局部构件或参数变换。旧反例用于回归检查，避免修改后的归约程序重复出现已经发现的错误。

反例搜索的目标是寻找满足下式的源实例：

$$\text{Format}_1(x) = 1 \quad \wedge \quad \left(\text{Format}_2(f(x)) = 0 \vee \text{oracle}_{L_1}(x) \neq \text{oracle}_{L_2}(f(x)) \right). \quad (4.4)$$

若 YES 实例被映射为 NO 实例，问题通常出现在证书保留或目标构造不足；若 NO 实例被映射为 YES 实例，问题通常出现在反向约束缺失。输出格式错误则先对应到目标实例构造。评测框架保存这些反馈，供智能体修正归约程序并开展回归检查。

组合任务还要保存中间实例的格式和规模界。若已知两步映射分别为 f 和 g ，复合映射为 $h(x) = g(f(x))$ 。当 $|f(x)| \leq p(|x|)$ 且 $|g(y)| \leq q(|y|)$ 时，输出长度受 $q(p(|x|))$ 控制。评测框架记录中间对象数量和数值位长，避免归约程序在组合后隐含指数规模输出。

归约程序通过有限实例检验后，还需要整理为数学证明。归约成立所需的证明内容见表 4.3。程序通过率只说明当前测试没有发现反例，不能替代表中任一证明。

表 4.3 归约成立所需的证明内容。

Table 4.3 Proof requirements for a reduction.

证明内容	主要结论	评测框架记录
目标实例构造	输出符合目标问题输入格式	格式检查结果、构造说明
正向蕴含	$x \in L_1 \Rightarrow f(x) \in L_2$	YES 反例、证书变换线索
反向蕴含	$f(x) \in L_2 \Rightarrow x \in L_1$	NO 反例、缺失约束记录
运行时间	构造步骤关于输入长度的上界	程序结构、循环边界
输出长度	对象数量与数值位长的增长上界	输出规模统计、中间实例记录

4.4 未知复杂度问题的 NPC 性证明尝试

前述 NPC 归约程序构造数据集、基础代码智能体和 NPC 归约评测框架均服务于提升人工智能辅助 NPC 证明的能力上限。已知归约任务用于评估和改进智能体的构造能力，实例反馈用于发现并修正归约映射中的错误，为未知问题的归约探索提供基础。

在此基础上，本课题将复杂度尚未确定的 NP 判定问题作为重点探索对象，尝试完成候选源问题选择、归约构造和数学证明整理，力争解决一个问题的 NPC 性判定。

4.5 少样本实例反馈案例

以带负整数的子集和问题到 0-1 背包判定问题的归约为例。源实例由整数列表 S 和目标值 T 组成，目标是判断是否存在子集，其元素之和等于 T 。目标实例记为 (W, V, C, P) ，分别表示物品重量、物品价值、背包容量和目标价值；当且仅当存在一组物品，使总重量不超过 C 且总价值不低于 P 时，目标实例为 YES。

单次生成设置下得到的归约证明实现如算法 1 所示。该实现直接令重量和价值等于源问题中的元素，并将 T 作为背包容量。

该归约证明实现只适用于元素非负的特殊情形。当 S 含有负数时， $W = S$ 会生成负重量，不符合背包问题的输入格式。该实现也没有输出目标价值 P ，因而没有完整定义目标判定条件。即使所有元素均为非负数，也需要设置 $P = T$ ，才能利用 $W = V$ 把总重量不超过 T 且总价值不低于 T 限定为子集和恰好等于 T 。

在少样本实例反馈设置下，评测框架返回了包含负元素的实例及判定不一

算法 1 单次生成设置下的归约证明实现

Require: 整数列表 S , 目标值 T **Ensure:** 重量 W , 价值 V , 容量 C 1: $S \leftarrow [\text{int}(s) \mid s \in S]$ 2: $T \leftarrow \text{int}(T)$ 3: $W \leftarrow S$ 4: $V \leftarrow S$ 5: $C \leftarrow T$ 6: **return** (W, V, C)

致信息。智能体据此生成归约证明实现，如算法 2 所示。该实现记所有负元素之和为 N ，把每个元素转换为绝对值，并将目标值平移为 $T' = T - N$ 。

算法 2 少样本实例反馈下的归约证明实现**Require:** 整数列表 S , 目标值 T **Ensure:** 重量 W , 价值 V , 容量 C , 目标价值 P

```

1:  $S \leftarrow [\text{int}(s) \mid s \in S]$ 
2:  $T \leftarrow \text{int}(T)$ 
3:  $N \leftarrow \sum_{s \in S, s < 0} s$ 
4:  $W \leftarrow [|s| \mid s \in S]$ 
5:  $V \leftarrow W$ 
6:  $T' \leftarrow T - N$ 
7: if  $T' < 0$  then
8:   return  $([1], [1], 0, 1)$ 
9: end if
10: if  $W = \emptyset$  then
11:   return  $([1], [1], 0, T')$ 
12: end if
13:  $C \leftarrow T'$ 
14:  $P \leftarrow T'$ 
15: return  $(W, V, C, P)$ 

```

该归约证明实现的关键是用负元素的补集表示原问题中的负数选择。设 A 为源实例选取的元素下标集合，在目标实例中选取所有属于 A 的非负元素，并选取所有不属于 A 的负元素所对应的绝对值，得到下标集合 B 。两者满足

$$\sum_{i \in B} |s_i| = \sum_{i \in A} s_i - N. \quad (4.5)$$

因此， $\sum_{i \in A} s_i = T$ 当且仅当 $\sum_{i \in B} |s_i| = T - N = T'$ 。设置 $W = V$ 且 $C = P = T'$ 后，背包约束要求所选物品的总重量恰好为 T' ，从而保持两个问题的 YES/NO 判定。若 $T' < 0$ ，非负重量之和不可能达到该值，该实现返回固定 NO 实例；空列表分支则保留空子集在 $T' = 0$ 时的判定结果。

该案例表明，少样本实例反馈暴露了单次生成中未覆盖的输入域和输出字段。修正后的归约证明实现不是针对某个反例调整常数，而是补充负元素变换、目标价值和边界实例处理，形成对任务输入范围有效的统一归约构造。

第 5 章 研究条件与工作计划

5.1 研究条件

目前已完成 NPC 归约程序构造数据集的基础建设，并基于 132 项任务集合完成消融实验。基础代码智能体与 NPC 归约评测框架已有初步实现，具备任务运行、归约程序执行和实例判定等研究基础。

后续研究需要稳定的大语言模型服务、隔离的代码执行环境，以及支持批量实验与判定程序运行的计算资源。现有开发环境与实验资源可支撑数据集完善、评测框架优化和归约案例研究。

5.2 研究工作计划

表 5.1 研究工作计划。

Table 5.1 Research schedule.

时间	主要工作
2026.07 至 2026.12	完成 NPC 归约程序构造数据集的构建
2027.01 至 2027.03	完成智能体相关基础设施的搭建和优化
2027.03	完善研究结果，补充实验，整理结论
2027.03 至 2027.04	撰写硕士毕业论文并准备参与预答辩
2027.05	递交硕士毕业论文材料，参加最终答辩

参考文献

- Azerbayev Z, Piotrowski B, Schoelkopf H, et al. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics [J/OL]. arXiv preprint arXiv:2302.12433, 2023. <https://arxiv.org/abs/2302.12433>.
- Cook S A. The complexity of theorem-proving procedures [C/OL]//Proceedings of the Third Annual ACM Symposium on Theory of Computing. Association for Computing Machinery, 1971: 151-158. DOI: [10.1145/800157.805047](https://doi.org/10.1145/800157.805047).
- Creus C, Fernández P, Godoy G. Automatic evaluation of reductions between np-complete problems [C/OL]//Theory and Applications of Satisfiability Testing. Springer, 2014: 415-421. DOI: [10.1007/978-3-319-09284-3_30](https://doi.org/10.1007/978-3-319-09284-3_30).
- DiCicco M, Worden E, Olsen C, et al. The karp dataset [J/OL]. arXiv preprint arXiv:2501.14705, 2025. <https://arxiv.org/abs/2501.14705>.
- Fan L, Hua W, Li L, et al. NPHardEval: Dynamic benchmark on reasoning ability of large language models via complexity classes [C/OL]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, 2024: 4092-4114. DOI: [10.18653/v1/2024.acl-long.225](https://doi.org/10.18653/v1/2024.acl-long.225).
- Garey M R, Johnson D S. Computers and intractability: A guide to the theory of np-completeness [M]. San Francisco: W. H. Freeman, 1979.
- Grange J, Vehlken F, Vortmeier N, et al. Specification and automatic verification of computational reductions [C/OL]//Leibniz International Proceedings in Informatics: volume 306 49th International Symposium on Mathematical Foundations of Computer Science. 2024: 56:1-56:14. DOI: [10.4230/LIPIcs.MFCS.2024.56](https://doi.org/10.4230/LIPIcs.MFCS.2024.56).
- Hubert T, Mehta R, Sartran L, et al. Olympiad-level formal mathematical reasoning with reinforcement learning [J/OL]. Nature, 2026, 651: 607-613. DOI: [10.1038/s41586-025-09833-y](https://doi.org/10.1038/s41586-025-09833-y).
- Jiang A Q, Welleck S, Zhou J P, et al. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs [C/OL]//International Conference on Learning Representations. 2023. <https://arxiv.org/abs/2210.12283>.
- Jimenez C E, Yang J, Wettig A, et al. SWE-bench: Can language models resolve real-world github issues? [C/OL]//International Conference on Learning Representations. 2024. <https://openreview.net/forum?id=VTF8yNQM66>.
- Karp R M. Reducibility among combinatorial problems [M/OL]//Miller R E, Thatcher J W. Complexity of Computer Computations. Plenum Press, 1972: 85-103. DOI: [10.1007/978-1-4684-2001-2_9](https://doi.org/10.1007/978-1-4684-2001-2_9).

- Lample G, Lacroix T, Lachaux M A, et al. Hypertree proof search for neural theorem proving [C]// Advances in Neural Information Processing Systems: volume 35. 2022: 26337-26349.
- Li X, Fang X, Ding S, et al. NP-Engine: Empowering optimization reasoning in large language models with verifiable synthetic np problems [J/OL]. arXiv preprint arXiv:2510.16476, 2025. <https://arxiv.org/abs/2510.16476>.
- Pan X W, An S W, Liu J G. Problem reductions at scale: Agentic integration of computationally hard problems [J/OL]. arXiv preprint arXiv:2604.11535, 2026. <https://arxiv.org/abs/2604.11535>.
- Romera-Paredes B, Barekatin M, Novikov A, et al. Mathematical discoveries from program search with large language models [J/OL]. Nature, 2024, 625: 468-475. DOI: [10.1038/s41586-023-06924-6](https://doi.org/10.1038/s41586-023-06924-6).
- Trinh T H, Wu Y, Le Q V, et al. Solving olympiad geometry without human demonstrations [J/OL]. Nature, 2024, 625: 476-482. DOI: [10.1038/s41586-023-06747-5](https://doi.org/10.1038/s41586-023-06747-5).
- Wang X, Li B, Song Y, et al. OpenHands: An open platform for ai software developers as generalist agents [J/OL]. arXiv preprint arXiv:2407.16741, 2024. <https://arxiv.org/abs/2407.16741>.
- Xin H, Ren Z Z, Song J, et al. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search [J/OL]. arXiv preprint arXiv:2408.08152, 2024. <https://arxiv.org/abs/2408.08152>.
- Yang C, Wang R, Jiang J, et al. Nondeterministic polynomial-time problem challenge: An ever-scaling reasoning benchmark for llms [J/OL]. arXiv preprint arXiv:2504.11239, 2025. <https://arxiv.org/abs/2504.11239>.
- Yang J, Jimenez C E, Wettig A, et al. SWE-agent: Agent-computer interfaces enable automated software engineering [C/OL]//Advances in Neural Information Processing Systems. 2024. <https://arxiv.org/abs/2405.15793>.
- Yang K, Swope A M, Gu A, et al. LeanDojo: Theorem proving with retrieval-augmented language models [C/OL]//Advances in Neural Information Processing Systems: volume 36. 2023. <https://arxiv.org/abs/2306.15626>.
- Yao S, Zhao J, Yu D, et al. ReAct: Synergizing reasoning and acting in language models [C/OL]// International Conference on Learning Representations. 2023. https://openreview.net/forum?id=WE_vluYUL-X.
- Zhang C, Hartline J D, Dimoulas C. Karp: A language for np reductions [C/OL]//Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation. Association for Computing Machinery, 2022: 762-776. DOI: [10.1145/3519939.3523732](https://doi.org/10.1145/3519939.3523732).
- Zheng K, Han J M, Polu S. miniF2F: A cross-system benchmark for formal olympiad-level mathe-

matics [C/OL]//International Conference on Learning Representations. 2022. <https://arxiv.org/abs/2109.00110>.